

INFO145 - Diseño y Análisis de Algoritmos

1 - Introducción

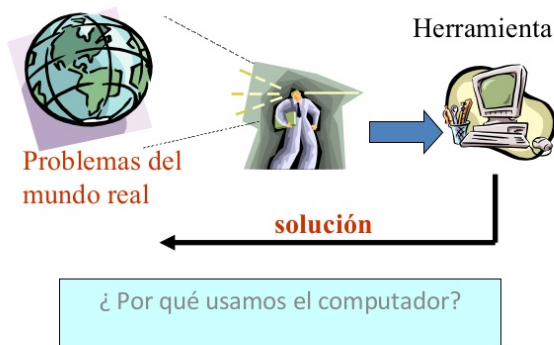
Héctor Ferrada

académico

Instituto de Informática
Universidad Austral de Chile

Problemas Tratables Computacionalmente

Entre todos los problemas existentes del mundo real, solo un pequeño número de ellos son **tratables** computacionalmente.



¿Cómo Abordar un Problema Computacionalmente?

Debemos definir un **Modelo de Cómputo** dónde un problema $\mathcal{P}$ será tratado, en donde:

- Nos permite definir el dominio (cuantitativo) de $\mathcal{P}$.
- Nos ofrece herramientas de transformación/cómputo sobre los datos y herramientas de validación.

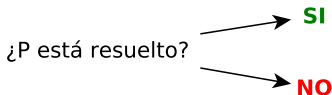
¿Cómo Abordar un Problema Computacionalmente?

Debemos definir un **Modelo de Cómputo** dónde un problema $\mathcal{P}$ será tratado, en donde:

- Nos permite definir el dominio (cuantitativo) de $\mathcal{P}$.
- Nos ofrece herramientas de transformación/cómputo sobre los datos y herramientas de validación.

Tal que podemos:

1. **Seleccionar** cualquier subconjunto (finito) del dominio
⇒ Datos de entrada a $\mathcal{P}$
2. **Aplicar** transformaciones sucesivas a los datos
⇒ **Algoritmo** que resuelve $\mathcal{P}$.
3. **Certificar** si un estado (conjunto de datos) es una solución a $\mathcal{P}$:



*$\mathcal{P}$ es un problema
de decisión*

¿Qué es un Algoritmo?

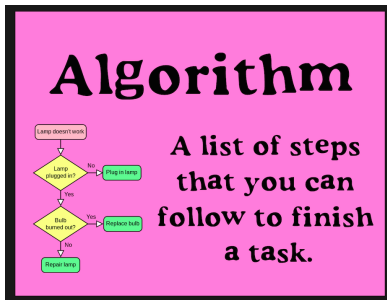
¿Qué es un Algoritmo?

Algorithm



**A list of steps
that you can
follow to finish
a task.**

¿Qué es un Algoritmo?



Un algoritmo es un proceso bien definido que puede tomar uno o varios valores de entrada (el *input*) y a través de acciones precisas, ordenadas y en pos de un objetivo concreto, producir un resultado de salida (el *output*).

Algoritmos Para Todo

Podemos encontrar algoritmos en todas las áreas de la ciencias de la computación; por ejemplo, algoritmos para:

- Optimización de problemas → TSP.
- Indexación de texto → Pattern Matching.
- Data compression → Texto, Imagen, Video, Números, etc.
- Big data → Reconocimiento de patrones en grandes volúmenes de datos.
- Data mining → Problemas de Clusterización (ejem. k-means)
- Computación de alto rendimiento → Paralelización en GPU/CPU
- ... y mucho más

¿Porqué Estudiar Algoritmos?

¿Porqué Estudiar Algoritmos?

- Importancia en todas las áreas de las ciencias de la computación → Los algoritmos están en el corazón de todos sus procesos
- Juegan un rol primordial en toda innovación tecnológica → Ejem. Web search engine
- Nos provee de un lenguaje formal para hablar acerca del comportamiento de un programa y una forma de describir sus acciones.
- Nos proveen de las herramientas y estrategias para evaluar diferentes soluciones a un mismo problema y dar una estimación cuantitativa del performance de una solución (análisis).

¿Qué es necesario para estudiar algoritmos (INFO145)?

- Tener un moderado dominio en programación → habrán tareas de implementación.
- Conocer el lenguaje de descripción en pseudocódigo → para describir los algoritmos que estudiemos.
- Dominio matemático básico → cursos introductorios de algebra y cálculo son más que suficiente.
- Dominio en estructuras de datos elementales: arreglos, listas enlazadas, pilas, colas, heaps, árboles, tablas de hash.
→ los algoritmos y las estructuras de datos son inseparables
- Prerequisitos: INFO085 - Estructuras de Datos y Algoritmos, BAIN091 - Estadística y Probabilidades para Ingeniería y INFO099 - Estructuras Discretas

El curso

Cubre dos áreas:

1. Análisis de algoritmos

- Estudio teórico, a priori, del performance de un programa (o segmento de código) al ejecutarlo en un computador.
- Por tanto existe un **Modelo de Cómputo bien definido**; es decir, un conjunto de reglas que respetar.

El curso

Cubre dos áreas:

1. Análisis de algoritmos

- Estudio teórico, a priori, del performance de un programa (o segmento de código) al ejecutarlo en un computador.
- Por tanto existe un **Modelo de Cómputo bien definido**; es decir, un conjunto de reglas que respetar.

2. Diseño de algoritmos

- Al Diseñar un algoritmo ¿Qué cosas son más importantes que el **performance** de este?

El curso

Cubre dos áreas:

1. Análisis de algoritmos

- Estudio teórico, a priori, del performance de un programa (o segmento de código) al ejecutarlo en un computador.
- Por tanto existe un **Modelo de Cómputo bien definido**; es decir, un conjunto de reglas que respetar.

2. Diseño de algoritmos

- Al Diseñar un algoritmo ¿Qué cosas son más importantes que el **performance** de este?
→ correctitud, simplicidad, estabilidad, seguridad, funcionalidad, localidad, mantenibilidad, ¿otras?.

El curso

Cubre dos áreas:

1. Análisis de algoritmos

- Estudio teórico, a priori, del performance de un programa (o segmento de código) al ejecutarlo en un computador.
- Por tanto existe un **Modelo de Cómputo bien definido**; es decir, un conjunto de reglas que respetar.

2. Diseño de algoritmos

- Al Diseñar un algoritmo ¿Qué cosas son más importantes que el **performance** de este?
→ correctitud, simplicidad, estabilidad, seguridad, funcionalidad, localidad, mantenibilidad, ¿otras?.

En este curso solo nos limitaremos a **estudiar el diseño y el análisis del performance teórico y empírico de algoritmos** (que asumimos correctos).

A esto también podríamos añadir un mayor dominio en ciertas estructuras de datos.

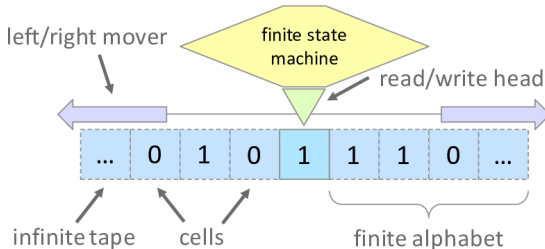
El Diseño y Análisis de Algoritmo

- Definimos como una **Instancia** de un problema $\mathcal{P}$, a una entrada válida a $\mathcal{P}$; y a un algoritmo $\mathcal{A}$ como **Correcto**, si este produce la salida correcta (output) para toda instancia posible a $\mathcal{P}$.
- Un algoritmo debe respetar las reglas impuestas por el **modelo de computo** en el cuál fue modelado el problema.
- El algoritmo debe ser preciso, no ambiguo, objetivo e indicar el orden de realización de cada paso.
- De importancia, para la mayor parte de este curso, son los **Algoritmos Determinísticos**. En estos, las acciones a ejecutar están bien definidas y determinadas; es decir, si este se ejecuta varias veces con la misma entrada, entonces se deben de ejecutar siempre las mismas acciones y obtener la misma salida.
- **Todo algoritmo debe ser finito**. Al seguir su flujo de datos e instrucciones se debe terminar en algún momento, o dicho de otro modo, debe tener un número finito de pasos.
- Todo algoritmo debe ser independiente del lenguaje de programación en el cuál se desee implementar y siempre se debe poder describir por medio de un **lenguaje de descripción de algoritmos**, como el **pseudocódigo** o DFD.

Máquina de Turing

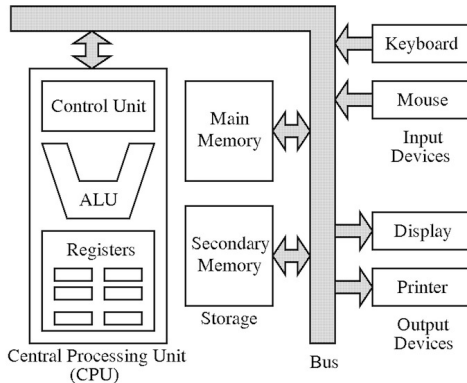
Para entender lo que es un **Módulo de Cómputo** (que es lo que nos interesa), debe tener en cuenta lo que es una Máquina de Turing y la Arquitectura de John Von Neumann:

- Una Máquina de Turing es una **máquina teórica** (un modelo matemático) que tiene una **memoria infinita** (un cinta) en la que es posible almacenar tanto datos como instrucciones, un cabezal para la lectura/escritura de símbolos, un registro de estados finitos y una tabla de instrucciones. Todo esto hace posible la creación y ejecución de programas.



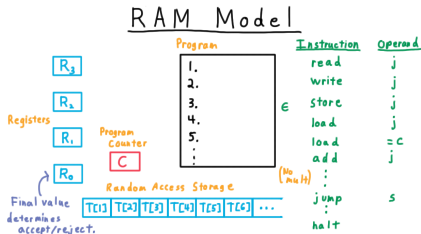
Arquitectura de John Von Neumann

- Es un modelo de arquitectura de computador que hace posible la ejecución de programas que previamente se almacenan en la memoria de la computadora.



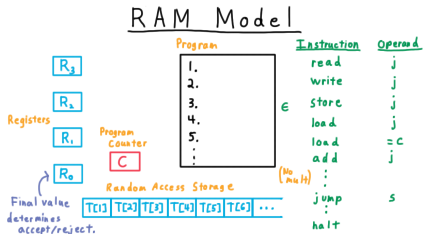
RAM - Nuestro Modelo de Cómputo

Un programa se escribe para ser interpretado por el computador, pero un algoritmo se escribe para que lo entiendan las personas, no las máquinas.



RAM - Nuestro Modelo de Cómputo

Un programa se escribe para ser interpretado por el computador, pero un algoritmo se escribe para que lo entiendan las personas, no las máquinas.



El Modelo RAM es un modelo matemático de computación que nos permite realizar una abstracción del problema a fin de analizar matemáticamente un algoritmo y comparar diferentes soluciones en términos de su rendimiento. En este modelo se asume que:

- Se cuenta con un único procesador y que las instrucciones son ejecutadas una después de la otra (secuencialidad) y no hay operaciones concurrentes.
- Existe una Unidad de Control en donde se carga un programa (algoritmo).
- La memoria es ilimitada y esta dividida en celdas (registros).

... Nuestro Modelo de Cómputo

- En cada registro de memoria hay un valor entero no negativo y de largo arbitrario.
- Al cargar un programa en la UC, este es enumerado como un conjunto de instrucciones.
- Existe un *Program Counter* que determina cual instrucción es la siguiente a ejecutar.
- Acceder a cualquier registro de memoria toma una unidad de tiempo.
- Las operaciones simples (+, *, =, *if*, *call*) toman 1 unidad de tiempo, donde los loops y subrutinas no son consideradas como operaciones simples.
- El tiempo de ejecución de un programa es asumido como la cantidad de unidades de tiempo que se ejecutan.
- El espacio requerido por un programa es la cantidad de registros de memoria que fueron requeridos.

Ejemplo Ordenamiento

Ejemplo de un problema del mundo real $\Rightarrow$ Ordenamiento basado en comparaciones de los elementos.

- **Problema $\mathcal{P}$ en el modelo** $\Rightarrow$ dada $S = \{s_1, s_2, \dots, s_n\}$, determinar una permutación S' de S , tal que: $S' = \{s'_1 \leq s'_2 \leq \dots \leq s'_n\}$.
- **Cota Inferior** $\rightarrow n \log n$ comparaciones (estimación del tiempo).

Ejemplo Ordenamiento

Ejemplo de un problema del mundo real $\Rightarrow$ Ordenamiento basado en comparaciones de los elementos.

- **Problema $\mathcal{P}$ en el modelo** $\Rightarrow$ dada $S = \{s_1, s_2, \dots, s_n\}$, determinar una permutación S' de S , tal que: $S' = \{s'_1 \leq s'_2 \leq \dots \leq s'_n\}$.
- **Cota Inferior** $\rightarrow n \log n$ comparaciones (estimación del tiempo).

Sorting Algorithms



- **Algoritmo** $\Rightarrow$ Insertionsort, quicksort, mergesort, heapsort, etc.
- **Cota Superior:** insertionsort, quicksort $\Rightarrow c_1 \cdot n^2$, c_1 constante.
mergesort, heapsort $\Rightarrow c_2 \cdot n \log n$, c_2 constante.

Ejemplo Ordenamiento

Ejemplo de un problema del mundo real $\Rightarrow$ Ordenamiento basado en comparaciones de los elementos.

- **Problema $\mathcal{P}$ en el modelo** $\Rightarrow$ dada $S = \{s_1, s_2, \dots, s_n\}$, determinar una permutación S' de S , tal que: $S' = \{s'_1 \leq s'_2 \leq \dots \leq s'_n\}$.
- **Cota Inferior** $\rightarrow n \log n$ comparaciones (estimación del tiempo).

Sorting Algorithms



- **Algoritmo** $\Rightarrow$ Insertionsort, quicksort, mergesort, heapsort, etc.
- **Cota Superior:** insertionsort, quicksort $\Rightarrow c_1 \cdot n^2$, c_1 constante.
mergesort, heapsort $\Rightarrow c_2 \cdot n \log n$, c_2 constante.
- **Performance Empírico:** quicksort es el más rápido para entradas grandes, mientras que insertionsort lo es para entradas relativamente pequeñas.

¿Cómo medimos el desempeño de un algoritmo?

¿Cómo medimos el desempeño de un algoritmo?

Por la cantidad de recursos que consume:

¿Cómo medimos el desempeño de un algoritmo?

Por la cantidad de recursos que consume:

- Tiempo de CPU
- Cantidad de comunicación.
- Número de acceso a disco (secuencial vs aleatorio)
- Cantidad de memoria principal

Principalmente nos enfocaremos en medir el **tiempo de CPU**.

Teniendo siempre en cuenta que también es muy importante:

1. **La cantidad de memoria** requerida
2. **El consumo energético** de sus implementaciones

¿Cómo medimos el desempeño de un algoritmo?

Por la cantidad de recursos que consume:

- Tiempo de CPU
- Cantidad de comunicación.
- Número de acceso a disco (secuencial vs aleatorio)
- Cantidad de memoria principal

Principalmente nos enfocaremos en medir el **tiempo de CPU**.

Teniendo siempre en cuenta que también es muy importante:

1. **La cantidad de memoria** requerida
2. **El consumo energético** de sus implementaciones

Considere los siguientes adagios en ciencias de la computación:

- *El 80% del tiempo de ejecución de un programa toma lugar en no más del 20% de su código*
- *Antes de implementar, asegúrese de tener un buen diseño*

Tiempo de procesamiento en CPU de un algoritmo

Consideraremos:

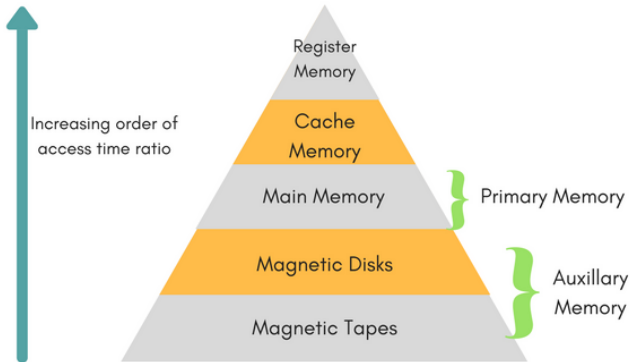
- La jerarquía de la memoria*
- Prefetching de instrucciones (carga de instrucciones en memoria rápida) y branch prediction (if-then-else: aprovecha las probabilidades)
- Pipeline (ocupar todas las parte del procesador en paralelo)

Tiempo de procesamiento en CPU de un algoritmo

Consideraremos:

- La jerarquía de la memoria*
- Prefetching de instrucciones (carga de instrucciones en memoria rápida) y branch prediction (if-then-else: aprovecha las probabilidades)
- Pipeline (ocupar todas las parte del procesador en paralelo)
- Pero por sobre todo lo anterior, nos enfocaremos en el **número de instrucciones** que se consideran relevantes (primitivas) en el análisis.
- Estas instrucciones primitivas son **representantes del tiempo real** que el algoritmo requiere.

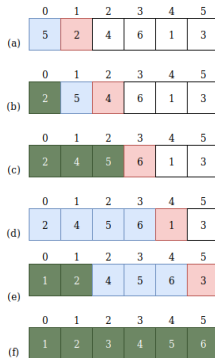
La Jerarquía de la Memoria



A medida que logremos que las estructuras de datos de nuestros algoritmos se carguen en niveles de memoria más altos, el algoritmo tendrá un mejor desempeño (tiempo de CPU).

Análisis del Pseudocódigo de un Algoritmo

Input: un arreglo de enteros $A[1..n]$
Output: A ordenado crecientemente.
insertionSort(A, n) {
 for $i = 2$ to n do
 $key = A[i]$
 $j = i - 1$
 while ($j > 0$ AND $A[j] > key$) do
 $A[j + 1] = A[j]$
 $j = j - 1$
 $A[j + 1] = key$
}



- Se busca una buena **cota superior** al algoritmo, para estimar el **tiempo teórico** (a priori) en el peor caso.
- Es equivalente al número de **operaciones relevantes** que se ejecutan para una instancia cualquiera, lo que nos da una relación directa con el **tiempo de ejecución** de $\mathcal{A}$ en el peor caso.

Algoritmo $\mathcal{A}$ que Encuentra el Mínimo - Pseudocódigo

Input: un arreglo de n elementos $A[1..n]$

Output: min , el elemento menor en A

```
minimo( $A, n$ ) {  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
}
```

- Podríamos definir que las **instrucciones relevantes son todas las de comparación**, es decir las del ciclo *for* y de la sentencia *if*.
- O Podríamos establecer que lo importante es la cantidad de veces que los elementos de A son comparados, es decir solo las de la sentencia *if*.

Algoritmo $\mathcal{A}$ que Encuentra el Mínimo - Pseudocódigo

Input: un arreglo de n elementos $A[1..n]$

Output: min , el elemento menor en A

```
minimo( $A, n$ ){  
     $min = A[1]$   
    for  $i = 2$  to  $n$  do  
        if ( $A[i] < min$ ) then  
             $min = A[i]$   
}
```

- Podríamos definir que las **instrucciones relevantes son todas las de comparación**, es decir las del ciclo *for* y de la sentencia *if*.
- O Podríamos establecer que lo importante es la cantidad de veces que los elementos de A son comparados, es decir solo las de la sentencia *if*.
- Lo que intereza es la **proporción** de instrucciones relevantes y no la cantidad exacta; es decir, nos interesa la **Tasa de Crecimiento del Algoritmo**